

轻量版 IT Support Ticket Assistant

Multi-Agent Practice Project 执行说明

用于 Claude Code Plan Mode + Codex Review/Project Management 的项目启动文档

0. 如何使用这份文档

把这份文档的内容作为 Claude Code Plan Mode 的项目上下文。目标不是让 agent 一次性做完所有功能，而是让 Claude Code 先做细致计划，再按阶段执行；Codex 主要负责 review、项目管理、任务拆分和风险控制。

- 第一轮：把本文档交给 Claude Code，并要求它只生成 plan，不要立即改代码。
- 第二轮：Claude Code 根据 plan 创建 docs/、CLAUDE.md、AGENTS.md、tasks.md 等项目管理文件。
- 第三轮：Claude Code 开始实现前端和轻量业务逻辑；每完成一个小阶段提交一次。
- 第四轮：把改动交给 Codex 做 code review、项目范围检查、bug 风险检查和下一步任务规划。

1. 项目概述

项目名称：Lightweight IT Support Ticket Assistant。

项目定位：一个适合新手练习 agent 开发的轻量 IT 工单管理 Web App。它模拟小型公司内部 IT Support 团队处理员工请求的流程，重点练习前端页面、数据建模、状态管理、AI 辅助回复和 agent 协作开发流程。

- 目标用户：IT Support / Helpdesk 人员、小型公司内部员工。
- 核心问题：员工 IT 请求分散在邮件、聊天和口头沟通中，难以追踪状态、优先级和处理记录。
- MVP 目标：能创建工单、查看列表、筛选状态/优先级、查看详情、添加回复/处理记录，并生成一个 AI suggested reply 的草稿。
- 练习目标：熟悉 Next.js + TypeScript 前端开发、组件拆分、mock data、项目文档化、agent 分工和 code review。

2. MVP 范围

模块	必须实现	暂不实现 / 后续扩展
Ticket Dashboard	工单统计卡片；按状态/优先级筛选；搜索标题/请求人；最近工单列表	复杂 BI 图表、多团队权限
Ticket List	显示 ID、标题、请求人、分类、优先级、状态、更新时间	服务器端分页、高级查询语法
Ticket Detail	显示工单详情、处理记录、内部备注、状态切换	真实邮件同步、附件上传
Create Ticket	员工姓名、邮箱、标题、描述、分类、优先级	完整用户登录系统、审批流
AI Suggested Reply	根据工单描述生成一段 mock AI 回复草稿，可复制/插入记录	真实 LLM API、自动发送邮件
Project Docs	README、CLAUDE.md、AGENTS.md、docs/tasks.md、docs/decisions.md	复杂架构文档平台

3. 推荐技术栈

- Framework: Next.js App Router。
- Language: TypeScript。
- Styling: Tailwind CSS。
- UI Components: shadcn/ui 或基础自定义组件。新手阶段可以先用自定义组件，避免一次引入太多依赖。
- Data: 第一阶段只使用 mock data，例如 src/lib/mock-tickets.ts。
- State: 先用 React useState/useMemo；后续再考虑 Zustand 或 server actions。
- Testing: 先做 basic lint/build check；后续再加 unit tests。
- Deployment: 后续可部署到 Vercel。MVP 阶段先确保本地 npm run dev 正常。

4. 页面和功能设计

4.1 Dashboard 页面

- 顶部统计: Open、In Progress、Waiting、Resolved 数量。
- 中间区域: 搜索框、状态筛选、优先级筛选。
- 主区域: Ticket table 或 card list。
- 右侧或下方: 最近更新、AI triage 提示、常见问题分类。

4.2 Ticket Detail 页面

- 标题、状态、优先级、分类、请求人信息。
- 问题描述。
- Activity timeline: 创建记录、状态变化、内部备注、回复记录。
- AI Suggested Reply 区块: 根据 ticket 内容生成模板回复。
- 操作按钮: Change status、Add internal note、Copy AI reply。

4.3 Create Ticket 页面

- Requester name / email。
- Issue title。
- Category: Email、Network、Hardware、Software、Access、Other。
- Priority: Low、Medium、High、Urgent。
- Description。
- Submit 后跳转到 ticket list 或 ticket detail。

4.4 Empty/Loading/Error 状态

- 没有工单时显示 friendly empty state。
- 筛选无结果时显示 clear filters 按钮。
- AI reply loading 使用 skeleton 或 simple loading text。
- 错误状态用可读提示，不要直接显示技术报错。

5. 数据模型建议

```
type TicketStatus = 'open' | 'in_progress' | 'waiting' | 'resolved';
type TicketPriority = 'low' | 'medium' | 'high' | 'urgent';
```

```
type TicketCategory = 'email' | 'network' | 'hardware' | 'software' | 'access' | 'other';

type Ticket = {
  id: string;
  title: string;
  requesterName: string;
  requesterEmail: string;
  category: TicketCategory;
  priority: TicketPriority;
  status: TicketStatus;
  description: string;
  createdAt: string;
  updatedAt: string;
  assignedTo?: string;
  activities: TicketActivity[];
};

type TicketActivity = {
  id: string;
  type: 'created' | 'status_changed' | 'note' | 'reply';
  author: string;
  content: string;
  createdAt: string;
};
```

6. 项目文件结构建议

```
lightweight-it-ticket-assistant/
├─ CLAUDE.md
├─ AGENTS.md
├─ README.md
├─ docs/
│  ├─ project-brief.md
│  ├─ frontend-spec.md
│  ├─ tasks.md
│  ├─ decisions.md
│  ├─ codex-review.md
│  └─ prompts/
│     ├─ claude-plan-mode-prompt.md
│     └─ codex-review-prompt.md
├─ src/
│  ├─ app/
│  │  ├─ page.tsx
│  │  └─ tickets/[id]/page.tsx
│  │     └─ tickets/new/page.tsx
│  ├─ components/
│  │  ├─ ticket-list.tsx
│  │  ├─ ticket-card.tsx
│  │  ├─ ticket-filters.tsx
│  │  ├─ ticket-detail.tsx
│  │  └─ ai-suggested-reply.tsx
│  └─ lib/
│     ├─ mock-tickets.ts
│     └─ ticket-utils.ts
```

7. Multi-Agent 分工方式

本项目不要让 Claude Code 和 Codex 互相自动调用。更稳定的方式是：同一个 Git repo，两套 agent 通过文档、任务清单、review 文件和 Git commit 交接。

Agent	主要职责	允许做的事	避免做的事
-------	------	-------	-------

Claude Code	前端设计 + 代码实现	生成 plan；创建页面；写组件；实现 mock data；更新 CLAUDE.md 和 docs/tasks.md	不要扩大 MVP 范围；不要一次性重构所有文件；不要接真实 API 除非明确要求
Codex	Code review + 项目管理	检查 bug；检查 TypeScript/lint/build；审查组件结构；更新 AGENTS.md；整理下一步任务	不要随意重新设计 UI；不要覆盖 Claude 的完整实现；不要未经确认引入大型依赖
User / Owner	最终决策	确认功能范围；运行项目；决定下一阶段是否扩展 backend/AI API	不要同时给两个 agent 下互相冲突的指令

8. Agent 协作协议

1. 所有 agent 开始前必须先读 README.md、docs/project-brief.md、docs/tasks.md。
2. Claude Code 每次实现功能前必须先写小计划，列出会修改哪些文件。
3. Claude Code 每完成一个阶段，必须更新 docs/tasks.md 的 Completed / Next / Risks。
4. Codex review 时只做 review 和小修复，优先输出 docs/codex-review.md。
5. 任何 agent 都不要一次性改动超过一个 feature area，例如不要同时改 Dashboard、Detail、Create Ticket 和全局样式。
6. 引入新依赖前必须写原因、替代方案和影响。
7. 如果 build/lint 失败，必须先修复失败，再继续新功能。

9. Claude Code Plan Mode Prompt

把下面这段复制给 Claude Code，建议先使用 plan mode。

You are the main frontend design and implementation agent for my practice project.

Project: Lightweight IT Support Ticket Assistant.
Goal: Build a beginner-friendly MVP web app for internal IT support ticket management.

Important workflow rules:

1. Start in plan mode. Do not modify files until the plan is clear.
2. Read and create project documents first.
3. Keep the MVP small and avoid over-engineering.
4. You are responsible for frontend design and implementation.
5. Codex will be responsible for code review and project management, so your work must be easy to review.

Please create a detailed implementation plan with phases:

- Phase 0: Project setup and docs
- Phase 1: Mock data and data model
- Phase 2: Dashboard and ticket list
- Phase 3: Ticket detail page
- Phase 4: Create ticket page
- Phase 5: AI suggested reply mock feature
- Phase 6: Polish, accessibility, README

For each phase, include:

- goal
- files to create or modify
- implementation steps
- acceptance criteria

- risks
- what Codex should review after the phase

Do not start coding until I approve the plan.

10. Codex Review / Project Manager Prompt

You are the code review and project management agent for this practice project.

Project: Lightweight IT Support Ticket Assistant.
Claude Code is responsible for frontend design and implementation.
Your responsibility is to keep the project clean, reviewable, and within MVP scope.

Please review the current repository and produce docs/codex-review.md.

- Check:
1. Does the implementation match docs/project-brief.md and docs/tasks.md?
 2. Are there TypeScript, lint, build, or runtime risks?
 3. Is the component structure clean and beginner-friendly?
 4. Is the MVP scope controlled, or did the project become too complex?
 5. Are there missing empty/loading/error states?
 6. Are accessibility basics covered?
 7. What should Claude Code do next?

- Rules:
- Do not redesign the whole UI.
 - Do not introduce large dependencies.
 - Prefer small, actionable review comments.
 - If you make code changes, keep them minimal and explain them clearly.
 - Update docs/tasks.md with Next Actions and Risks.

11. 阶段任务清单

阶段	任务	验收标准
Phase 0 - Setup and Docs	初始化 Next.js 项目；创建 README.md、CLAUDE.md、AGENTS.md、docs/project-brief.md、docs/tasks.md。	能运行 npm run dev；docs 文件存在；范围清晰。
Phase 1 - Data Model	创建 Ticket 类型、mock ticket 数据、ticket utils。	页面可读取 mock data；类型无明显 any 滥用。
Phase 2 - Dashboard/List	实现首页统计、筛选、搜索、ticket list。	能按状态/优先级筛选；搜索可用；空状态可用。
Phase 3 - Detail Page	实现 /tickets/[id] 页面、timeline、status badge、activity list。	点击 ticket 能进入详情；无效 id 有友好提示。
Phase 4 - Create Ticket	实现 /tickets/new 表单；提交后模拟创建或显示成功状态。	表单字段清楚；基本校验可用；不需要真实数据库。
Phase 5 - AI Suggested Reply	根据 ticket 描述生成 mock reply template。	有 loading/生成/复制状态；明确标注是 suggested draft。
Phase 6 - Polish and Review	优化 responsive、accessibility、README、review notes。	本地运行正常；docs/tasks.md 记录下一步扩展。

12. CLAUDE.md 建议内容

CLAUDE.md

Role

Claude Code is the frontend design and implementation agent.

Project Goal

Build a lightweight IT Support Ticket Assistant MVP for practice.

Rules

- Start with planning before coding.
- Keep the MVP small and beginner-friendly.
- Prefer simple components over complex abstractions.
- Use mock data before real backend integration.
- Update docs/tasks.md after each phase.
- Do not introduce new dependencies without explaining why.
- Make code easy for Codex to review.

Required Output After Each Phase

- Summary of completed work
- Files changed
- How to run/test
- Known issues
- Suggested Codex review focus

13. AGENTS.md 建议内容

AGENTS.md

Repository Expectations

- Read docs/project-brief.md and docs/tasks.md before making changes.
- Keep changes small and focused.
- Run lint/build checks when available.
- Update docs/tasks.md after completing review or implementation work.

Codex Role

Codex acts as code review and project management agent.

Review Checklist

- MVP scope control
- TypeScript safety
- component clarity
- no unnecessary dependency
- responsive layout
- empty/loading/error states
- accessibility basics
- README accuracy

Handoff Protocol

Claude Code implements features.

Codex reviews and writes docs/codex-review.md.

The user decides which recommended fixes to apply.

14. Definition of Done

- App can run locally without obvious errors.
- Dashboard, list, detail, create ticket and mock AI reply are usable.
- Code is TypeScript-based and component structure is understandable.
- README explains setup, features, and limitations.
- docs/tasks.md shows completed tasks, next tasks, and risks.
- Codex review file exists and major high-priority issues are fixed or documented.
- MVP has not expanded into a complex enterprise helpdesk system.

15. 后展方向

- 加入 Supabase / Firebase 存储真实 tickets。
- 加入登录和角色权限：employee / IT support / admin。
- 接入真实 AI API 生成回复、分类和优先级建议。
- 把邮件内容导入成工单。
- 加入 SLA、due date、assignee 和 notification。
- 加入 dashboard analytics。

16. 参考说明

Claude Code 支持 plan mode，常见用法包括使用 `--permission-mode plan` 或在会话中切换 plan mode；Claude Code 也支持通过 CLAUDE.md 保存项目记忆和指令。Codex 支持通过 AGENTS.md 为项目提供持久指导，适合存放 review 规则、build/test 命令和项目约定。

- Anthropic Claude Code docs: common workflows, memory, subagents.
- OpenAI Codex docs: AGENTS.md project guidance, customization, Codex CLI, subagents.

End of Project Plan Brief